

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

ERROR ANALYSIS TASK

Name of the Student	Guggilla Venkatsai
Reg. No	192425388

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I Guggilla Venkatsai (192425388) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



QUESTIONS:

Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

- Identify the conceptual error in the likelihood formulation.**

Answer:

The conceptual error is that the student writes the likelihood function as the sum of probabilities instead of the product of probabilities. In Maximum Likelihood Estimation (MLE), each

observation is assumed to be independent. Therefore, the probability of observing the entire dataset is obtained by multiplying the probabilities of all observations, not by adding them. Using the sum of probabilities does not represent the joint probability of the data and violates the basic principle of probability for independent events. As a result, the likelihood function becomes mathematically incorrect, leading to inaccurate parameter estimation and poor model performance.

The correct likelihood function is:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

where:

- $L(\theta)$ = Likelihood function
- θ = Model parameter
- $P(x_i | \theta)$ = Probability of the i^{th} observation
- n = Total number of observations

b) Explain how this mistake affects parameter estimation.

Answer:

When the likelihood function is written as the sum of probabilities, the optimization algorithm does not maximize the true probability of the observed data. Instead, it searches for parameter values based on an incorrect mathematical function. As a result, the estimated parameters are inaccurate and fail to represent the actual data distribution.

This mistake reduces the classifier's prediction accuracy, produces poor generalization on unseen data, and may lead to incorrect decision boundaries. Even if a large amount of training data is available, the model cannot learn the correct parameters because the objective function itself is wrong.

c) Suggest the correct mathematical formulation and reasoning.

Answer:

The correct mathematical formulation of the likelihood function is:

$$L(\theta) = \prod_{i=1}^n P(x_i | \theta)$$

Since all observations are assumed to be independent, the probability of observing the complete dataset is equal to the product of the probabilities of each individual observation.

For optimization, MLE usually uses the log-likelihood:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

This formulation preserves the same maximum value while making calculations much simpler and more efficient.

d) How would using log-likelihood fix computational issues?

Answer:

The log-likelihood converts the product of many probabilities into a sum of logarithms. This avoids numerical underflow, which occurs when multiplying many very small probabilities together. It also simplifies differentiation and optimization, making gradient-based algorithms faster and more stable.

The log-likelihood is given by:

$$\log L(\theta) = \sum_{i=1}^n \log P(x_i | \theta)$$

Therefore, most machine learning algorithms maximize the log-likelihood instead of the likelihood because it is computationally efficient, mathematically simpler, and produces the same optimal parameter estimates.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

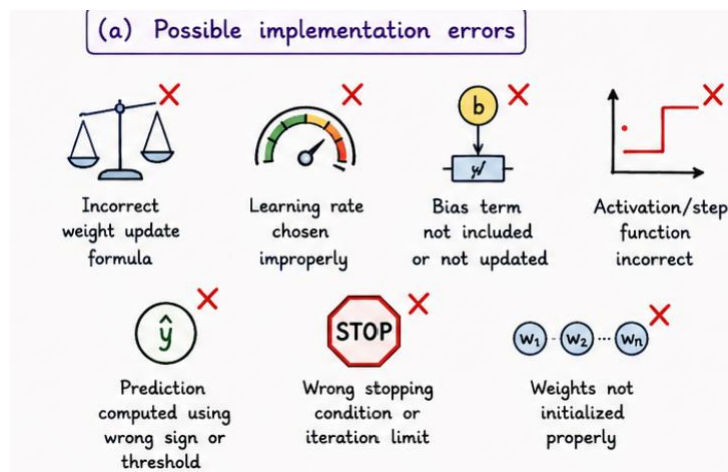
Tasks:

a) List possible implementation errors in the learning algorithm.

Answer:

A Perceptron may fail to converge due to several implementation errors in the learning algorithm. Common errors include using an incorrect weight update formula, choosing an inappropriate learning rate, not updating the bias term, or implementing the activation function incorrectly. Another common mistake is using the wrong stopping condition or incorrectly calculating the prediction before updating the weights.

These implementation errors prevent the Perceptron from learning the correct decision boundary, even when the dataset is linearly separable.



b) Analyze how incorrect learning rate or weight updates affect convergence.

Answer:

The learning rate determines the size of each weight update during training. If the learning rate is too high, the weights may overshoot the optimal solution, causing the model to oscillate and fail to converge. If the learning rate is too low, the updates become very small, making the training process extremely slow.

Similarly, if the weight update rule is implemented incorrectly, the weights move in the wrong direction instead of reducing classification errors. This prevents the Perceptron from finding the correct separating hyperplane, leading to poor convergence and low prediction accuracy.

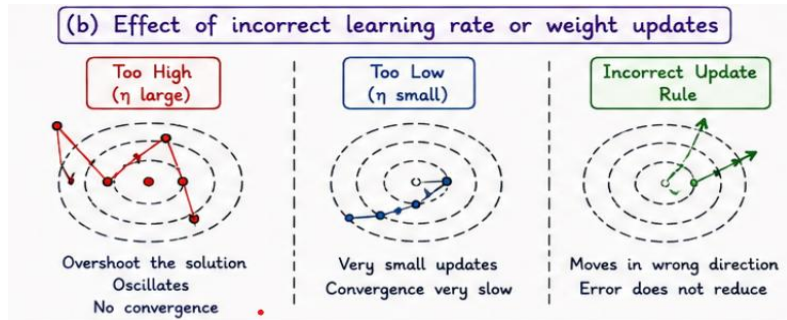
Example Formula

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} + \eta(\mathbf{y} - \hat{\mathbf{y}})\mathbf{x}$$

where:

- w = Weight
- η = Learning rate

- y = Actual output
- $\hat{y}$ = Predicted output
- x = Input

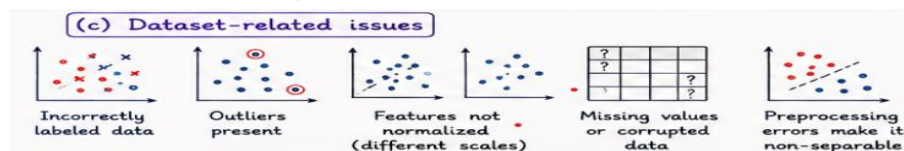


c) Identify dataset-related issues that may cause this problem.

Answer:

Dataset-related issues can also prevent convergence. The dataset may contain incorrectly labeled samples, noisy data, or missing values. Features with different scales can make learning difficult if they are not normalized. Although the problem states that the dataset is linearly separable, implementation errors in preprocessing or incorrect data loading may make it appear otherwise.

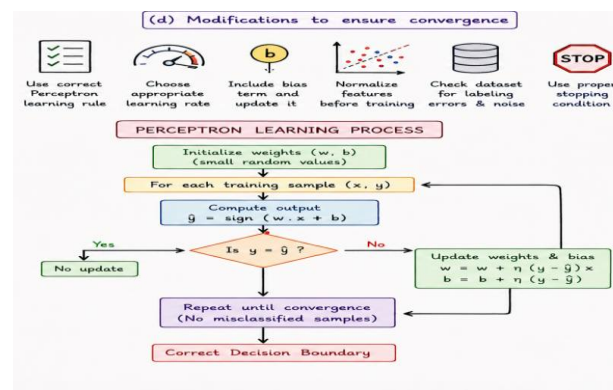
Therefore, proper data cleaning, normalization, and verification of labels are essential before training the Perceptron.



d) Propose modifications to ensure convergence.

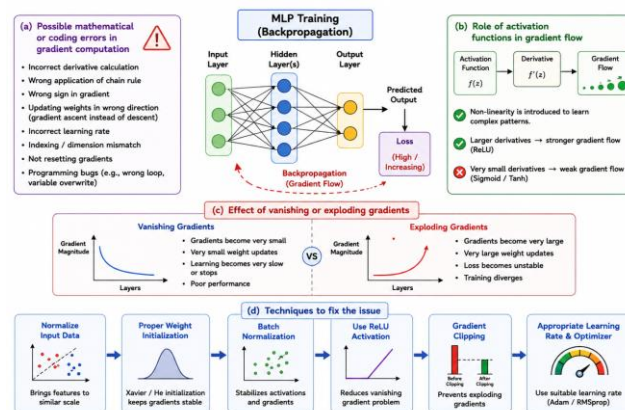
Answer:

To ensure convergence, the correct Perceptron learning rule should be implemented with an appropriate learning rate. The input features should be normalized, and the bias term should be included in every weight update. The dataset should be checked for labeling errors and noise before training. Proper stopping criteria and sufficient training iterations should also be used. By applying these modifications, the Perceptron can successfully learn the correct decision boundary and achieve convergence on a linearly separable dataset.



3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.



Tasks:

a) Identify possible mathematical or coding errors in gradient computation.

Answer:

A Multilayer Perceptron (MLP) may show increasing loss if there are errors in the gradient computation during backpropagation. Common mathematical or coding errors include incorrect calculation of derivatives, improper implementation of the chain rule, updating weights in the wrong direction, incorrect learning rate, and programming mistakes such as indexing or sign errors. These mistakes produce incorrect gradients, causing the model to move away from the optimal solution instead of minimizing the loss.

b) Explain the role of activation functions in gradient flow.

Answer:

Activation functions introduce non-linearity into a neural network, allowing it to learn complex patterns. During backpropagation, the derivative of the activation function determines how gradients are passed from one layer to another. If the derivative is very small (as in sigmoid or tanh for large inputs), the gradients become weak, slowing down learning. Activation functions like ReLU help maintain stronger gradients and improve training efficiency in deep neural networks.

c) Analyze how vanishing or exploding gradients affect training.

Answer:

In deep neural networks, gradients may become extremely small (vanishing gradients) or extremely large (exploding gradients) during backpropagation. Vanishing gradients result in very small weight updates, causing the network to learn slowly or stop learning altogether. Exploding gradients produce excessively large weight updates, leading to unstable training,

oscillating loss values, and failure to converge. Both problems reduce the model's accuracy and make optimization difficult.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer:

Several techniques can improve training stability and prevent gradient-related problems:

- Normalize input data before training.
- Use Xavier or He weight initialization.
- Apply Batch Normalization to stabilize activations.
- Use ReLU activation instead of sigmoid where appropriate.
- Apply Gradient Clipping to prevent exploding gradients.
- Choose an appropriate learning rate and optimizer (such as Adam).

These methods improve gradient flow, stabilize learning, reduce training loss, and help the neural network converge efficiently.

4. Gradient Descent vs SGD Error Investigation

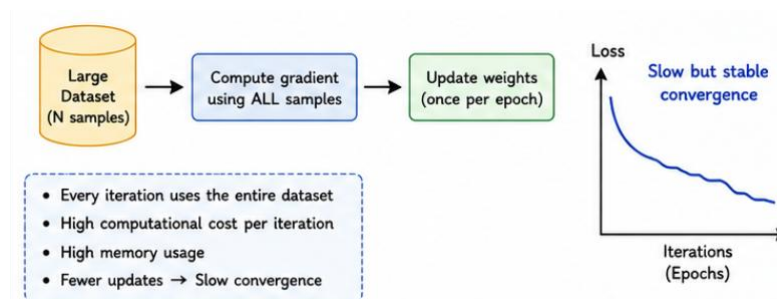
A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why Batch Gradient Descent is slow in large datasets.

Answer:

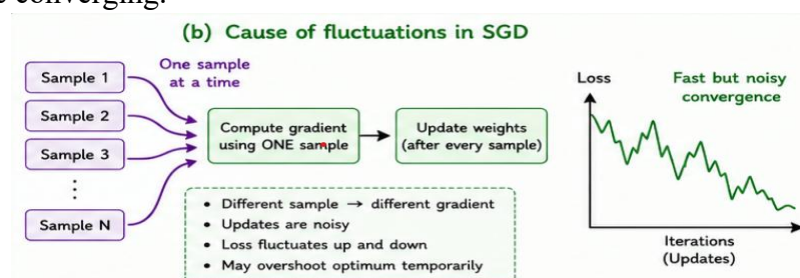
Batch Gradient Descent (BGD) updates the model weights only after processing the entire training dataset. When the dataset is very large, every iteration requires computing the gradients for all training samples. This increases computational time and memory usage, making the training process slow. As a result, the model takes longer to converge even though the updates are stable and accurate.



b) Analyze the cause of fluctuations in SGD.

Answer:

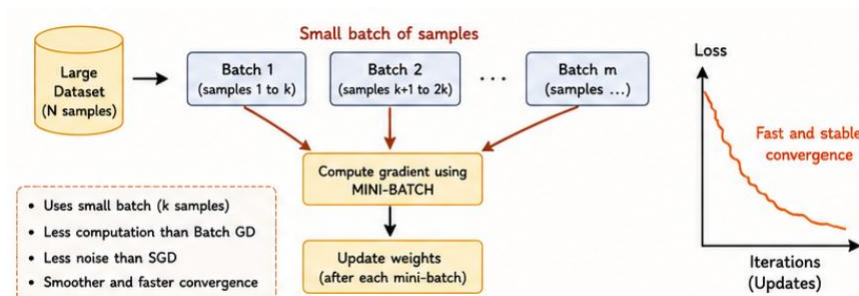
Stochastic Gradient Descent (SGD) updates the model weights using one training sample at a time. Since each sample is different, the calculated gradient changes continuously. This causes the loss curve to fluctuate instead of decreasing smoothly. Although SGD converges faster than Batch Gradient Descent, its updates are noisy and may temporarily move away from the optimum before converging.



c) Compare the error behavior of Mini-Batch Gradient Descent.

Answer:

Mini-Batch Gradient Descent combines the advantages of Batch Gradient Descent and SGD. It updates the weights using a small batch of samples (e.g., 32 or 64). The loss decreases more smoothly than SGD while requiring much less computation than Batch Gradient Descent. Therefore, Mini-Batch Gradient Descent provides faster training, lower fluctuations, and more stable convergence.



d) Recommend the best approach and justify with error trade-offs.

Answer:

Mini-Batch Gradient Descent is generally the best choice because it provides a good balance between speed and stability. It reduces the high computational cost of Batch Gradient Descent and minimizes the noisy fluctuations of SGD. As a result, it converges faster, uses memory efficiently, and achieves better generalization on large datasets. Therefore, Mini-Batch Gradient Descent is the most widely used optimization technique in modern deep learning.

	Batch GD	SGD	Mini-Batch GD
Speed	Slow	Fast	Fast
Stability	High (Stable)	Low (Noisy)	High (More Stable)
Memory Usage	High	Very Low	Moderate
Loss Behavior	Smooth	Fluctuates	Smoother
Best For	Small Datasets	Online / Real-time Learning	Large Datasets (General Use)

★ Mini-Batch Gradient Descent provides the best trade-off between speed and stability → Most widely used in practice.

5. Curse of Dimensionality Error Analysis

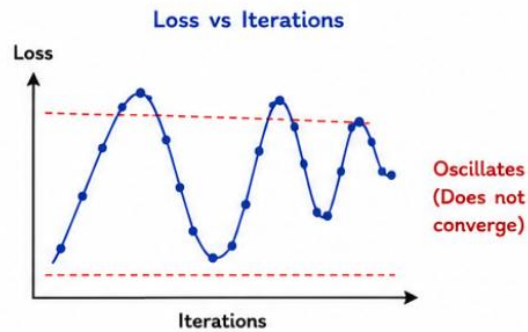
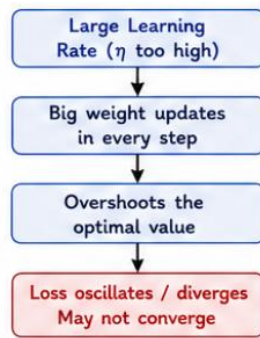
A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

Answer:

High dimensionality means the dataset contains a very large number of features. As the number of features increases, the data becomes sparse, making it difficult for the model to identify meaningful patterns. The model starts learning unnecessary details and noise from the training data instead of the actual relationships. This leads to overfitting, where the model performs well on training data but poorly on unseen test data. Therefore, high dimensionality increases prediction error and reduces the model's generalization ability.



b) Identify signs of overfitting in this scenario.

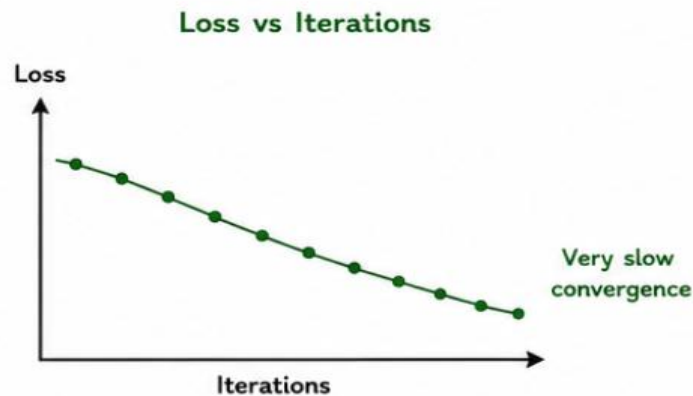
Answer:

Overfitting occurs when the model memorizes the training data instead of learning general patterns.

The main signs of overfitting are:

- Very high training accuracy.
- Low testing or validation accuracy.
- Large difference between training and testing performance.
- High prediction error on unseen data.
- Poor generalization to new datasets.

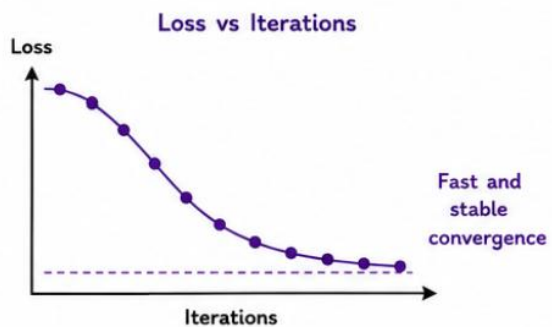
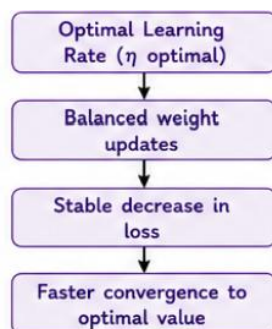
These signs indicate that the model has become too complex because of the large number of features.



c) Analyze why distance-based learning fails in high dimensions.

Answer:

Distance-based algorithms such as K-Nearest Neighbors (KNN) rely on the distance between data points. In high-dimensional spaces, most data points become almost equally distant from one another. This makes it difficult to identify the true nearest neighbors, reducing the effectiveness of distance calculations. As a result, the classifier produces inaccurate predictions and the overall model performance decreases.



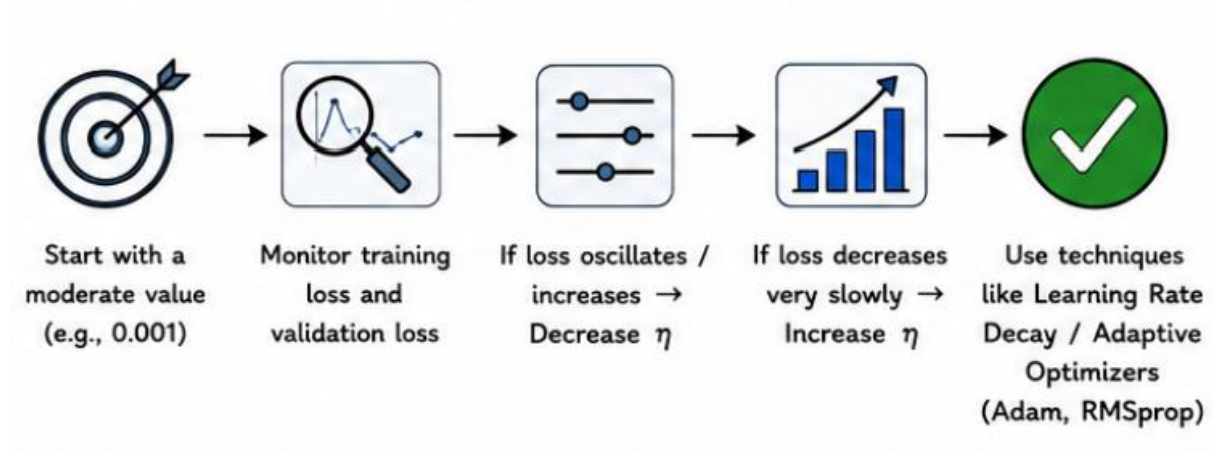
d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answer:

Several techniques can reduce the effects of the Curse of Dimensionality and improve model performance:

- **Principal Component Analysis (PCA):** Reduces the number of features while preserving important information.
- **Feature Selection:** Removes irrelevant and redundant features.
- **L1 (Lasso) Regularization:** Eliminates less important features by shrinking their weights to zero.
- **L2 (Ridge) Regularization:** Prevents overfitting by reducing large weight values.
- **Cross-Validation:** Helps select a model that generalizes well to unseen data.

These techniques reduce overfitting, improve generalization, and increase prediction accuracy.



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand